LAB 4 — PRACTICAL REPORT

Smart Optimization Assistant using Dynamic Programming

Name	Harinand S Parayil	**Roll No.**	2401201056
Specialisation	BCA — Artificial Intelligence and Data Science	**Section**	A

1. Aim and problem statement

To design and implement a Smart Optimization Assistant using Dynamic Programming for sequence analysis, resource allocation, and computation optimization. The application implements Longest Common Subsequence (LCS), 0/1 Knapsack, and Matrix Chain Multiplication. It displays DP tables and optimal solutions and compares theoretical and measured performance.

2. Core Dynamic Programming concepts

Optimal substructure: An optimal solution can be constructed from optimal solutions to smaller subproblems. LCS uses solutions for shorter prefixes, Knapsack uses best results for smaller item sets and capacities, and Matrix Chain uses optimal results for smaller matrix intervals.

Overlapping subproblems: Straightforward recursive solutions may solve the same smaller subproblems repeatedly. Dynamic Programming stores each required state and reuses it. This can be done using memoization (top-down) or tabulation (bottom-up).

DP tables: Bottom-up tabulation fills states in dependency order. The tables make intermediate results visible and show how the final optimum is reached.

3. Question 1 — Project workspace setup

The Python project workspace is organized with a source module, Streamlit interface, test file, requirements file, result data, screenshots, and documentation. A virtual environment isolates dependencies. Setup commands are provided in README.md.

4. Question 2 — Sequence analysis using LCS

LCS finds the longest sequence appearing in both strings in the same relative order, though not necessarily contiguously. For strings of lengths m and n, matching characters use the diagonal state plus one; otherwise the maximum of the upper and left states is selected. Example: ABCBDAB and BDCABA give length 4; BCBA is one valid LCS. Time: O(mn); space: O(mn).

5. Question 3 — Resource allocation using 0/1 Knapsack

0/1 Knapsack selects whole items to maximize total value without exceeding capacity. For each item and capacity, the algorithm compares excluding the item with including it when it fits. Example: weights [2, 3, 4, 5], values [3, 4, 5, 6], capacity 5 give maximum profit 7, selecting 1-based items [1, 2]. Time: O(nW); space: O(nW). The bound is pseudo-polynomial because it depends on the numeric capacity W.

6. Question 4 — Computation optimization using Matrix Chain Multiplication

Matrix Chain Multiplication determines the parenthesization minimizing scalar multiplication count; it does not multiply the matrices. The cost table stores minimum interval costs and the split table stores best split positions. Dimensions [10, 30, 5, 60] give minimum cost 4,500 and order ((A1 × A2) × A3). Time: $O(n^3)$; space: $O(n^2)$.

7. Question 5 — Theoretical complexity analysis

The table summarizes the asymptotic time and space bounds and the number of states or transitions represented by the standard tabulation algorithms.

Algorithm	Time complexity	Space complexity	DP subproblems / states
Longest Common Subsequence	O(m × n)	O(m × n)	(m+1)(n+1) table cells; m×n transitions
0/1 Knapsack	O(n × W)	O(n × W)	(n+1)(W+1) table cells; n×(W+1) transitions

Matrix Chain Multiplication	$O(n^3)$	$O(n^2)$	$O(n^2)$ interval states; $O(n^3)$ split evaluations

8. Question 5 — Measured performance analysis

The supplied complexity-comparison document reports the following largest-tested-input measurements. They are experimental observations from its preparation environment; runtime and traced memory vary with hardware, Python version, and implementation.

Algorithm	Largest tested input	Measured runtime	Peak traced memory	Recorded DP table cells
0/1 Knapsack	60 (input-size label)	0.007788 s	296.29 KiB	14,701
LCS	60 (input-size label)	0.000657 s	31.42 KiB	3,721
Matrix Chain	30 matrices	0.016635 s	30.52 KiB	961

The runtime trends should be compared against mn for LCS, nW for Knapsack, and n^3 for Matrix Chain. The DP table counts are consistent with the stated table dimensions in the supplied comparison. Peak traced Python memory is not the same as total process RAM. These sample measurements should be rerun on the submission computer if the lab requires locally collected results.

9. Question 6 — DP table visualizations

The following figures reproduce the DP table visualizations from the supplied visualization document. The original document header contained another student's identifying details; those details have not been carried into this report. The figures are included as supplied lab material.

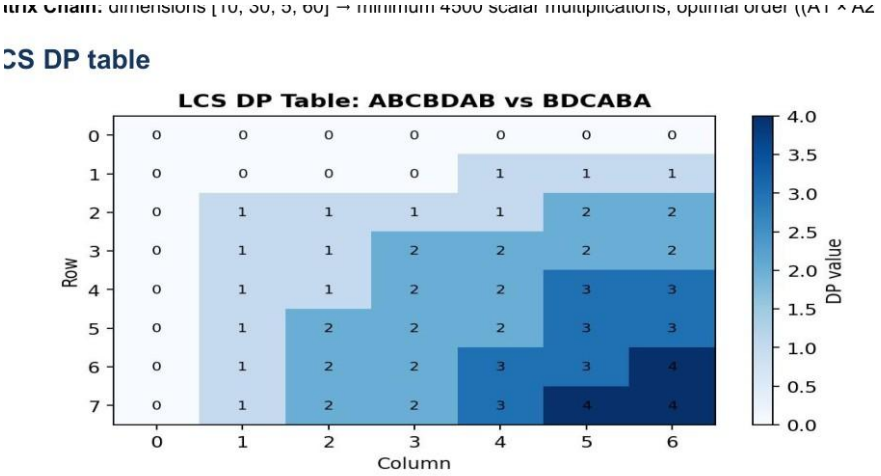


Figure 1. LCS DP table for ABCBDAB and BDCABA; the final table value is 4.

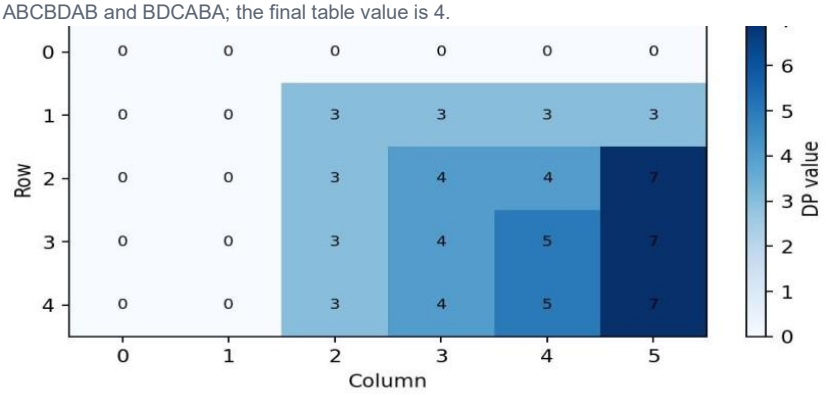


Figure 2. 0/1 Knapsack DP table for capacity 5; the maximum value is 7.

9. Question 6 — DP table visualizations (continued)

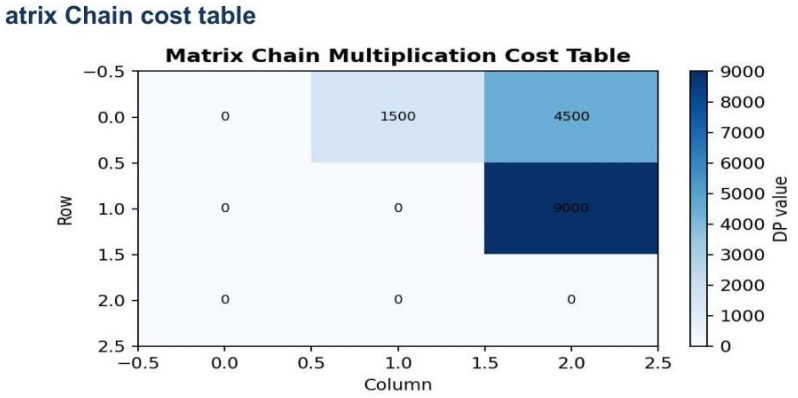


Figure 3. Matrix Chain minimum-cost table for dimensions [10, 30, 5, 60].

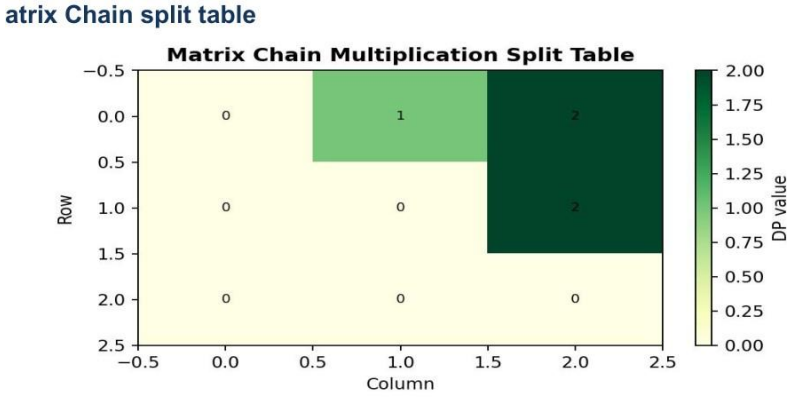




Figure 4. Matrix Chain split table showing the selected split positions.

10. Runtime and memory graphs

The graphs below show the measured runtime trends and peak traced Python memory reported in the supplied visualization document. The x-axis input-size labels are algorithm-specific, so they should not be interpreted as directly equivalent workloads across algorithms.

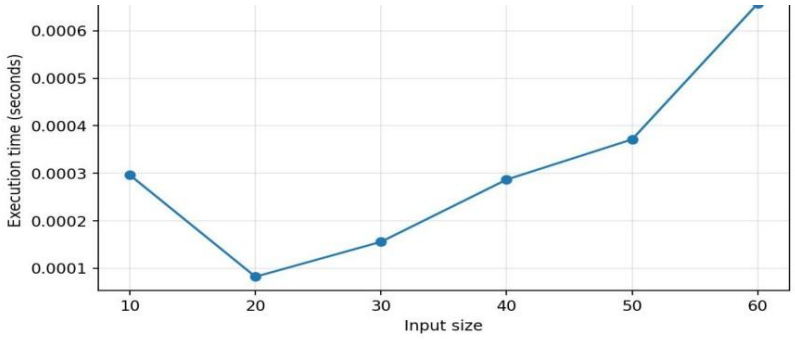


Figure 5. LCS measured runtime versus input size.

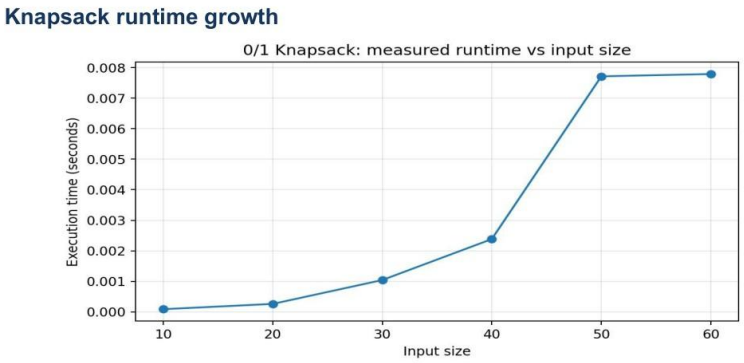


Figure 6. 0/1 Knapsack measured runtime versus input size.

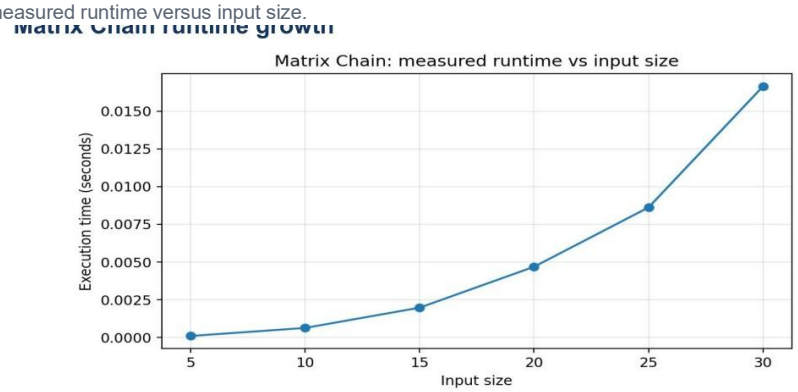




Figure 7. Matrix Chain measured runtime versus input size.

10. Runtime and memory graphs (continued)

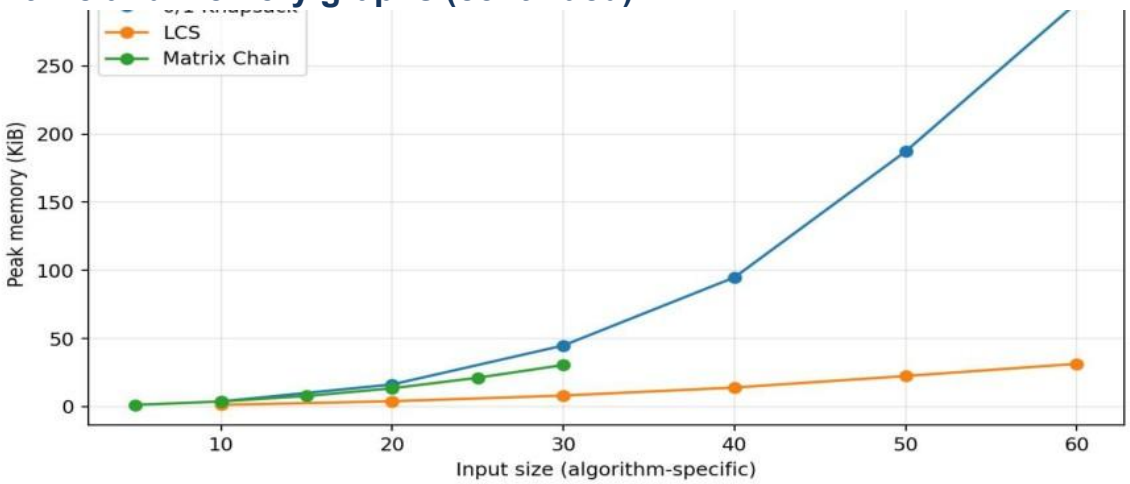


Figure 8. Peak traced Python memory by algorithm; this is not total operating-system process memory.

11. Key experimental observations

• LCS computes a rectangular table. Its state count grows with the product of the two sequence lengths.
• 0/1 Knapsack runtime depends on both item count and capacity; its O(nW) bound is pseudo-polynomial in numeric capacity
W.
• Matrix Chain Multiplication stores $O(n^2)$ interval states and examines possible split positions, giving $O(n^3)$ time.
• Traced memory is a Python-level measurement and is not identical to total process RAM.
• Measured values are hardware- and interpreter-dependent; the supplied CSV/plots represent an experimental sample, not auniversal benchmark.

12. Comparison with naive recursive approaches

Problem	Naive recursive approach	Dynamic Programming approach
LCS	Can take exponential time because prefix subproblems are recomputed repeatedly.	Stores prefix-pair states and runs in O(mn) time.
0/1 Knapsack	Explores include/exclude choices and can take O(2■) time.	Stores item-and-capacity states and runs in O(nW) time.
Matrix Chain	Explores parenthesizations with Catalan-number growth.	Stores interval results and evaluates $O(n^3)$ split choices.

This mirrors the Fibonacci comparison: naive recursion repeats work, while Dynamic Programming stores results so each required state is computed once.

13. Practical applications

LCS: Text comparison and version-control diff tools use common subsequences to identify shared ordered content. In bioinformatics, LCS can help compare DNA or protein sequences.

0/1 Knapsack: Models budgeting, project selection, and resource-constrained planning where each option has a value and a cost and the total budget or capacity is limited.

Matrix Chain Multiplication: Helps choose efficient orders for matrix operations in numerical computing, compiler optimization, and graphics or data-processing pipelines. Different parenthesizations yield the same mathematical result but can require different numbers of scalar operations.

14. Conclusion

The Smart Optimization Assistant demonstrates three classic Dynamic Programming techniques for sequence analysis, resource allocation, and computation optimization. It computes optimal results, exposes intermediate DP states, and compares theoretical complexity with experimental runtime and memory observations. The visualizations show how DP stores and reuses intermediate results to avoid repeated work.

15. GitHub repository

Repository URL: [https://github.com/Harinand07/Algorithm-Performance-Measurement-and-Benchmarking-Tool/tree/main/Lab4_Smart_Optimization_Assistant/Lab4_Smart_Optimization_Assistant]

Appendix — Sample execution

```
SMART OPTIMIZATION ASSISTANT — SAMPLE RUN

LCS input: ABCBDAB / BDCABA

LCS length: 4 | One valid LCS: BCBA

Knapsack weights: [2, 3, 4, 5], values: [3, 4, 5, 6], capacity: 5

Maximum profit: 7 | Selected item indices (0-based): [0, 1]

Matrix dimensions: [10, 30, 5, 60]

Minimum scalar multiplications: 4500
```

```
Optimal order: ((A1 × A2) × A3)

All three algorithms completed successfully.
```

Before submission: Enter your section and GitHub repository URL. Confirm the benchmark measurements against your own generated CSV or rerun the experiments on your computer if required by your instructor.